

```

/**
 * phantom-users-and-hardcoded-blocks.patch.ts
 *
 * Fix #2 – Third Unknown User `54374239` Consuming Resources
 * Fix #6 – `Jrt9VPmojMA` Has a Hardcoded Boot Block
 *
 * =====
 * FIX #2 – PHANTOM USER CLEANUP
 * =====
 */

// — Step A: Database cleanup (run once in Neon console or via migration) —
//
// This deletes or deactivates the phantom users so they stop burning resources.
// Check what's in these records before deleting – replace DELETE with
// UPDATE ... SET active = false if you want to soft-delete instead.
//
// RUN IN NEON CONSOLE:
//
// -- Inspect first
// SELECT id, email, created_at FROM users
// WHERE id IN ('54374239', 'tiktok_-000hfXLzkfKJGE24wvR-qZP9Pw6iwxLWyeM');
//
// -- If safe to delete (no important data):
// DELETE FROM users
// WHERE id IN ('54374239', 'tiktok_-000hfXLzkfKJGE24wvR-qZP9Pw6iwxLWyeM');
//
// -- OR soft-delete (safer):
// UPDATE users SET active = false
// WHERE id IN ('54374239', 'tiktok_-000hfXLzkfKJGE24wvR-qZP9Pw6iwxLWyeM');

// — Step B: Add a code-level guard in ALL per-user AI loops —
//
// After the DB cleanup, add this guard to prevent any future phantom users
// from consuming resources. Add to the top of every per-user processing loop:

const ACTIVE_PLATFORM_PREFIX_PATTERN = /^[0-9a-f]{8}-[0-9a-f]{4}/i; // UUID format
const PHANTOM_USER_IDS = new Set([
  'tiktok_-000hfXLzkfKJGE24wvR-qZP9Pw6iwxLWyeM',
  '54374239',
  // Add any other known phantom IDs here
]);

export function isActiveYouTubeUser(userId: string): boolean {
  // Reject known phantom IDs
  if (PHANTOM_USER_IDS.has(userId)) return false;
  // Reject non-YouTube platform prefixes

```

```

    if (userId.startsWith('tiktok_')) return false;
    if (userId.startsWith('rumble_')) return false;
    if (userId.startsWith('kick_')) return false;
    if (userId.startsWith('twitch_')) return false;
    // Accept UUID-format real user IDs
    return ACTIVE_PLATFORM_PREFIX_PATTERN.test(userId);
}

// Usage in all per-user loops (growth-flywheel, infinite-evolution,
// self-improvement-engine, consistency-agent, stream-lifecycle, etc.):
//
//   for (const userId of userIds) {
//     if (!isActiveYouTubeUser(userId)) {
//       log.debug(`Skipping non-active user: ${userId}`);
//       continue;
//     }
//     await runCycleForUser(userId);
//   }

// =====
// FIX #6 – REMOVE HARDCODED BOOT BLOCK FOR Jrt9VPmojMA
// =====
//
// The log shows:
//   [autopilot] Publish failed: "Cross-contaminated metadata –
//   permanently inaccessible (hardcoded boot block) (Jrt9VPmojMA)"
//
// The phrase "hardcoded boot block" means there's a literal string or array
// in the codebase that lists this video ID as permanently blocked.
//
// STEP 1: Find it
//   Search the codebase for: Jrt9VPmojMA
//   It will be in one of these patterns:
//
//   const PERMANENTLY_BLOCKED = ['Jrt9VPmojMA', ...];
//   // OR
//   if (videoId === 'Jrt9VPmojMA') throw new Error('hardcoded boot block');
//   // OR
//   const BLOCKED_VIDEOS = { 'Jrt9VPmojMA': 'Cross-contaminated metadata' };
//
// STEP 2: Remove the hardcode, replace with DB lookup
//   Instead of a hardcoded list, query the DB:

import { storage } from '../storage.js';
import { createLogger } from '../lib/logger.js';

const blockedLog = createLogger('autopilot');

/**

```

```

* Replace any hardcoded video block check with this DB-backed version.
* If a video needs to be blocked, mark it in the DB with a reason.
* The Resurrection Engine will eventually retry it after a cooldown.
*
* BEFORE (hardcoded):
*   const PERMANENTLY_BLOCKED = ['Jrt9VPmojMA'];
*   if (PERMANENTLY_BLOCKED.includes(videoId)) {
*     throw new Error('permanently inaccessible (hardcoded boot block)');
*   }
*
* AFTER (DB-backed, allows resurrection):
*/

export async function checkVideoBlockStatus(videoId: string): Promise<{
  blocked: boolean;
  reason: string | null;
}> {
  try {
    const block = await storage.getVideoBlock(videoId);
    if (!block) return { blocked: false, reason: null };

    blockedLog.warn(
      `[Autopilot] Video ${videoId} is blocked: ${block.reason} ` +
      `(resurrection_count: ${block.resurrectionCount}, ` +
      `retry_after: ${block.retryAfter?.toISOString() ?? 'none'})`
    );

    return { blocked: true, reason: block.reason };
  } catch {
    return { blocked: false, reason: null }; // fail open
  }
}

// STEP 3: Migrate the hardcoded block to the DB
// Run this once to move Jrt9VPmojMA from code to DB:
//
// INSERT INTO content_vault_backups (youtube_id, status, error_message, resurrection_count)
// VALUES ('Jrt9VPmojMA', 'failed', 'Cross-contaminated metadata', 0)
// ON CONFLICT (youtube_id) DO UPDATE
//   SET status = 'failed',
//       error_message = 'Cross-contaminated metadata',
//       resurrection_count = 0;
//
// The Resurrection Engine will then pick it up and retry after 48h (unknown error type).
// If it fails again, it will escalate the cooldown. Eventually it will either succeed
// or be archived after 20 resurrections – but NEVER permanently blocked in code.

```